

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 15: Design Patterns

Course taught by: Zubair Irshad



Logistics

1. Last lab quiz i.e. quiz 04 tomorrow i.e. Thursday!
 1. Topics: Inheritance, Polymorphism and Virtual Functions
 2. For Practice,
2. Plan for final project demos:
 1. We will use next Wednesday's class and Thursday's lab to take demos
 2. On Wednesday we will have group presentations, make sure to prepare a few slides, not needed. You will have 6 minutes to do the demo in class and 2 minutes for questions
 3. On Thursday, we will do individual questions/go over the code in detail.
3. Next Monday: Post-midterm Recap!



Unit Outline

- What is a Design Pattern?
- Categories of Patterns (Creational + Structural + Behavioral)
- In-depth: Selected Creational Patterns
- In-depth: Selected Structural Patterns
- In-depth: Selected Behavioral Patterns
- Summary + When to Use Which Pattern
- Q&A / Discussion



Motivation

- A design pattern is a *generic, repeatable solution* to a common problem in software design.
- Benefits: common vocabulary ("Factory", "Singleton", etc), facilitates communication among developers.
- We'll cover major categories, pick representative patterns, examine pros/cons, see UML/code.
- Also pitfalls: over-use can cause unnecessary complexity, hidden dependencies.
- Have you heard of code-refactoring? Design patterns can be useful for that as well!



Categories of Patterns

- **We see three broad categories:**
- **Creational:** Patterns centered around object creation (how objects are instantiated).
- **Structural:** Patterns about how classes/objects are composed into larger structures.
- **Behavioral:** Patterns about how objects interact / communicate.
- We'll pick ~2 from each category for deeper discussion.



Creational Design Pattern

- Creational design patterns provide **various object creation mechanisms**, which increase flexibility and reuse of existing code.
- Creational patterns help decouple the code that uses objects from the code that creates objects, improving maintainability.
- **When to use:** You don't want your client code to depend directly on concrete classes; you may want to vary what is created; you want to manage lifecycle.
- Two Examples: Factory Pattern, Singleton Pattern



Factory Design Pattern

- **Factory Method** is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.
- Instead of creating objects directly using **new** in main, you define a method (called a factory method) to create objects.
- This gives you flexibility and reduces tight coupling between code that uses the object and the code that creates it.
- Let's look at an example!



Example of Factory Design Pattern!

```
class Shape {  
public:  
    virtual void draw() = 0; // Pure virtual function  
    virtual ~Shape() {}  
};  
class Circle : public Shape {  
public:  
    void draw() override {  
        std::cout << "Drawing Circle\n";  
    }  
};  
class Square : public Shape {  
public:  
    void draw() override {  
        std::cout << "Drawing Square\n";  
    }  
};
```




Example of Factory Design Pattern!

```
class ShapeFactory {  
public:  
    virtual Shape* createShape() = 0; // Factory method  
    virtual ~ShapeFactory() {}  
};  
class CircleFactory : public ShapeFactory {  
public:  
    Shape* createShape() override {  
        return new Circle();  
    };  
};  
class SquareFactory : public ShapeFactory {  
public:  
    Shape* createShape() override {  
        return new Square();  
    };  
};
```



Example of Factory Design Pattern!

```
int main() {  
    ShapeFactory* factory;  
  
    factory = new CircleFactory();  
    Shape* shape1 = factory->createShape();  
    shape1->draw(); // Output: Drawing Circle  
    delete shape1;  
    delete factory;  
  
    factory = new SquareFactory();  
    Shape* shape2 = factory->createShape();  
    shape2->draw(); // Output: Drawing Square  
    delete shape2;  
    delete factory;  
    return 0;  
}
```



Factory Design – When to Use!

- You want to hide creation logic from the client.
- You have multiple types of related objects to create.
- You need to decide object type at runtime.
- Your object creation code may change or grow in the future.
- **Done need it when:**
 - Object creation is simple and not likely to change.
 - You don't need to hide or encapsulate object creation.



Singleton Design Pattern

- Ensures a class has only one instance
- The instance provides a global access point.
- Use cases: logger, configuration manager, database connection manager.



Single Design Pattern Example

```
class Logger {  
public:  
    static Logger& getInstance() {  
        static Logger instance;  // created once, thread-safe in C++11  
        return instance;  
    }  
  
    void log(const std::string& msg) {  
        std::cout << "[LOG] " << msg << std::endl;  
    }  
  
private:  
    Logger() = default;};
```



Single Design Pattern Example

```
// Example modules using the same logger
class NetworkModule {
public:
    void connect() {
        Logger::getInstance().log("Network
connected successfully.");
    }
};
```



Structural Patterns – Motivation!

- Structural patterns are about how classes/objects are composed into larger structures (e.g., object hierarchies, tree structures, wrappers).
- They help manage large codebases by organizing relationships, promoting reuse, and hiding complex subsystems.
- **When to use:** you have complicated object relationships, you need to adapt interfaces, you need to compose objects into part-whole hierarchies.
- **Two examples:** Adapter Pattern and Composite Pattern



Adapter Pattern

- **Adapter Pattern** is a structural design pattern used to make two incompatible interfaces work together.
- It acts as a bridge between two incompatible interfaces, allowing them to collaborate without modifying their source code.
- This pattern is particularly useful when integrating legacy code or third-party libraries into your application.



Adapter Design Pattern Example!

```
class SDCard {  
    int data;  
public:  
    SDCard(int d): data(d) {}  
    int SDCard_Transmit() {  
        cout << "Transmit data from SDCard\n";  
        return data;}};
```

```
class USB {  
protected: int data;  
public:  
    USB(int d): data(d) {}  
    virtual int USB_Transmit() {  
        cout << "Transmit data from USB\n";  
        return data;}};
```



Adapter Design Pattern Example!

```
// --- Client code ---  
class DataReader {  
public:  
    void ReadData(USB* device) {  
        int data = device->USB_Transmit();  
        cout << "Reader received data: " << data <<  
    }  
};
```



Adapter Design Pattern Example!

- **Problem:** DataReader only understands the **USB interface** (`USB_Transmit()`), but you have an **SDCard** that uses a different method (`SDCard_Transmit()`).
- **Solution:** Create an **Adapter** (`SDCard_USB_Adapter`) that:
 - Inherits from USB (so the client accepts it),
 - Contains an SDCard object and redirects `USB_Transmit()` → `SDCard_Transmit()`.
- **Analogy:** A **USB-to-SD card reader** — lets an SD card plug into a USB port.



Adapter Design Pattern Example!

```
// Solution create an Adapter: SDCard behaves like USB
class SDCard_USB_Adapter : public USB {
    SDCard* sdcard;
public:
    SDCard_USB_Adapter(SDCard* s): USB(-1), sdcard(s)
    {}
    int USB_Transmit() override { // override USB
interface
        return sdcard->SDCard_Transmit();
    };
};
```



Adapter Design Pattern Example!

```
int main() {  
    DataReader reader;  
    USB usb(5);  
    SDCard sd(10);  
  
    reader.ReadData(&usb);           // Works  
    // reader.ReadData(&sd);         // Error (incompatible)  
  
    SDCard_USB_Adapter adapter(&sd); // Fix with adapter  
    reader.ReadData(&adapter);  
}
```



Composite Design Pattern

- Compose objects into tree structures to represent part-whole hierarchies.
- In the case of photoshop where we draw many individual objects and then those objects compose a whole unique object.
 - Sometime, you might want to apply some operation on the whole object instead of the each of the individual objects.



Composite Design Pattern Example

```
// Base component
class PageObject {
public:
    virtual void display() = 0;
    virtual ~PageObject() = default;
};

// Leaf: individual page
class Page : public PageObject {
    string name;
public:
    Page(string n): name(n) {}
    void display() override {
        cout << "Page: " << name << endl;}};
```



Composite Design Pattern Example

```
// Composite: collection of pages
class Copy : public PageObject {
    vector<PageObject*> pages;
public:
    void add(PageObject* p) { pages.push_back(p); }

    void display() {
        cout << "Copy contains:\n";
        for (int i = 0; i < pages.size(); i++)
            pages[i]->display();
    }
};
```




Composite Design Pattern Example

```
int main() {  
    Page page1("Intro"), page2("Summary");  
    Copy copy;  
  
    copy.add(&page1);  
    copy.add(&page2);  
  
    copy.display();    // Treats all pages uniformly  
}
```



Creational Design Pattern

- Creational design patterns provide **various object creation mechanisms**, which increase flexibility and reuse of existing code.
- Creational patterns help decouple the code that uses objects from the code that creates objects, improving maintainability.
- **When to use:** You don't want your client code to depend directly on concrete classes; you may want to vary what is created; you want to manage lifecycle.
- Two Examples: Factory Pattern, Singleton Pattern



Structural Patterns – Motivation!

- Structural patterns are about how classes/objects are composed into larger structures (e.g., object hierarchies, tree structures, wrappers).
- They help manage large codebases by organizing relationships, promoting reuse, and hiding complex subsystems.
- **When to use:** you have complicated object relationships, you need to adapt interfaces, you need to compose objects into part-whole hierarchies.
- **Two examples:** Adapter Pattern and Composite Pattern



Behavioral Design Pattern

- Behavioral patterns focus on object interaction and responsibility assignment: how objects communicate, how algorithms vary, how responsibilities shift.
- **When to use:** you have varying algorithms, you need to decouple sender/receiver, you want to support undo/redo, you want to encapsulate change in object behavior.



Observer Design Pattern

- The Observer Pattern is a behavioral design pattern where:
 - One object (Subject) holds the main data.
 - Many other objects (Observers) want to be notified whenever that data changes.
 - This is called a "one-to-many" relationship.
- **Example: Weather Station & Displays**
- Let's say we have a Weather Station that collects temperature, humidity, and pressure.
- Main Idea: One object (**Subject**) notifies many others (**Observers**) when its data changes.



Observer Design Pattern Example

```
class Observer { // Observer Interface
public:
    virtual void update(float temp) = 0;};

class WeatherStation { // Subject (Weather Station)
Knows its observers and notifies them of changes
    vector<Observer*> observers;
    float temperature;
public:
    void addObserver(Observer* o) { observers.push_back(o); }
    void setTemperature(float t) {
        temperature = t;
        for (auto* o : observers) o->update(temperature);
    };
};
```



Observer Design Pattern Example

```
// Concrete Observer (Display)
class Display : public Observer {
    string name;
public:
    Display(string n): name(n) {}
    void update(float temp) override {
        cout << name << ": Temp = " << temp << "°C\
n";
    }
};
```



Observer Design Pattern Example

```
// CLIENT CODE
//      - Creates one Subject and multiple Observers.
//      - Registers Observers → when Subject changes, they auto-update.
int main() {
    WeatherStation station;           // Subject
    Display d1("Display1"), d2("Display2"); // Observers

    station.addObserver(&d1);
    station.addObserver(&d2);

    station.setTemperature(25.5);      // all observers auto-notified
    station.setTemperature(24.8);

}
```




Summary of Design Patterns Studied today!

Pattern	Category	When to Use	Main Pros	Main Cons
Factory Method	Creational	Need to create objects but hide concrete types	Loose coupling, extensible	Many classes, slightly more complex
Singleton	Creational	One and only one instance needed	Global access, controlled creation	Hidden dependencies, global state
Adapter	Structural	Need to make two interfaces work together	Reuse existing code	Extra layer, may hurt readability
Composite	Structural	Treat single objects and groups uniformly (e.g., files & folders)	Can make design abstract, harder to debug	Can make design abstract, harder to debug
Observer	Behavioral	One-to-many dependency updates	Decouples subject/observers	Memory/notification management



Which Pattern to choose!

- Ask questions like:
 - Do I need to control how objects are created? → Creational patterns.
 - Do I need to change object composition or interfaces? → Structural patterns.
 - Do I need to manage interactions, communication, or algorithmic variation? → Behavioral patterns.
- Avoid overuse: patterns are tools, not goals. Use them when they solve a problem, not just to use a fancy pattern.
- Often, patterns are combined in real systems (e.g., an Observer that uses a Factory to create observers).



Many more examples!

- Some more resources to read up on design patterns:
 - <https://refactoring.guru/design-patterns>
 - <https://www.geeksforgeeks.org/system-design/modern-c-design-patterns-tutorial/>



Design Patterns Exercise!

- Divide into groups and **pick one of the patterns**. It'd be great if you pick one not deeply covered today (e.g., Builder, Proxy, Flyweight, State, Template Method) from here:
<https://refactoring.guru/design-patterns>
- Discuss/Brainstorm the following:
 - Identify a real-world scenario (in games, UI, robotics, etc) where the pattern would apply.
 - Sketch a UML or code snippet for it.
 - List pros & cons in that scenario.



Conclusion

- Design patterns provide **reusable design solutions**, not finished code.
- Know the three categories: Creational, Structural, Behavioral.
- Each pattern has **trade-offs**: benefits and costs.
- Good design = pick the right pattern when needed, avoid over-engineering.
- Practice by identifying patterns in existing code and refactoring designs to use them when appropriate.